MÓDULO 3 · CURSO IAC UCM

Valores por entorno y el lock del estado

AGENDA

1 · Ficheros tfvars

1. **Ficheros tfvars**
2. Gestión de entornos
3. Lock del estado

Ficheros tfvars

El `.tf` no lleva lo que cambia

`.tf`

Recursos. El diseño del servicio.

`.tfvars`

Identificador, tamaño, correo, nombre del bucket.

Mismo flujo

`init`, `plan`, `apply`. Cambia el valor, no el procedimiento.

Ejemplo de tres variables sin `default`

En el laboratorio final van en `terraform.tfvars`. Pero hay más opciones, cada una con su caso de uso.

variables.tf

```
variable "identificador_alumno" { type = string }
variable "bucket_datos"         { type = string }
variable "correo_aviso"         { type = string }
```

terraform.tfvars

```
identificador_alumno = "ana_garcia"
bucket_datos         = "ucm-ana-garcia-menus"
correo_aviso         = "ana@ejemplo.com"
```

> Se sube a Git. En un proyecto real el `.tfvars` viaja con el código, para que el equipo aplique los mismos valores.

Diferentes opciones para el `.tfvars`

Fichero	¿Se carga solo?	Cuándo usarlo
`terraform.tfvars`	Sí	Un solo despliegue. Valores que siempre aplican en ese directorio.
`terraform.tfvars.json`	Sí, después del anterior	Lo mismo, cuando los valores salen en JSON.
`*.auto.tfvars`	Sí, por orden alfabético	Valores fijos que deben entrar siempre, sin flag.
`dev.tfvars`, `pro.tfvars`	No. Hace falta `-var-file`	Elegir el entorno en el comando.

> También se puede indicar una variable suelta, sin fichero: `terraform apply -var="numero_tareas=2"`.

Comandos para cada caso

Se cargan solos

- `terraform.tfvars`, `terraform.tfvars.json` y `*.auto.tfvars`
- Entran sin flag. Si hay varios `.auto`, entran todos

```
terraform apply
```

Elegir un fichero

- Ej.: `dev.tfvars` o `pro.tfvars`
- Se pasa con `-var-file`
- Se pueden pasar multiples `-var-file`

```
terraform apply -var-file="dev.tfvars"
```

```
terraform apply -var-file="dev.tfvars" -var-file="dev.local.tfvars"
```

> El `apply` tiene que llevar el mismo `-var-file` que el `plan`, para que tenga coherencia.

Quién pisa a quién

Gana la fuente de más abajo. El `default` solo entra si nadie ha asignado la variable.

	Fuente
1	`default` del bloque `variable`
2	Variable de entorno `TF_VAR_` + el nombre
3	`terraform.tfvars`
4	`terraform.tfvars.json`
5	`*.auto.tfvars`, por orden alfabético
6	`-var` y `-var-file`. El de la derecha gana

AGENDA

2 · Gestión de entornos

1. Ficheros tfvars
2. **Gestión de entornos**
3. Lock del estado

Para qué es cada entorno

Entorno	Aquí	Qué cambia
`dev`	Probar el cambio	Sufijo `-dev`, un recurso pequeño
`pre`	Ensayar con datos parecidos a producción	Sufijo `-pre`, tamaño igual o uno por debajo
`pro`	Lo que usan otros	Nombres estables, más tareas

> En los entornos laborales es muy común tener 3, o al menos 2 entornos. De esta manera se pueden hacer pruebas sin afectar al entorno productivo.

Multiples entorno implica multiples estados

ERRÓNEO · EL MISMO ESTADO

1

apply dev

Crea `ucm-taller-dev-menus`. El estado lo recuerda.

›

2

apply pro

Mismo directorio. Destruye el bucket de dev y crea el de pro.

CORRECTO · UN ESTADO POR ENTORNO

1

apply dev

Key `…/dev/terraform.tfstate`. Crea el bucket de dev.

+

2

apply pro

Key `…/pro/terraform.tfstate`. El de dev sigue.

El mismo `main.tf`, tres ficheros

Sirve cuando los tres entornos replican los mismos recursos, cada uno con una configuración distinta.

dev.tfvars

```
env = "dev"
```

pre.tfvars

```
env = "pre"
```

pro.tfvars

```
env = "pro"
```

```
resource "aws_s3_bucket" "menus" {
  bucket = "ucm-taller-${var.env}-menus"
}
```

> Se puede usar el argumento `count` de los recursos para desplegar o no un recurso en un entorno.

Cada entorno tiene su carpeta

Sin `-var-file`. Los valores van en el `default` de su `variables.tf`.

dev/

pre/

pro/

> Cada uno tendra X ficheros `.tf` con su configuraciones, `variables.tf`, etc.

Cuándo copiar los recursos

Ventajas

- Los entornos tienen recursos distintos
- Equipo de varias personas sobre el mismo tfstate

Desventajas

- Tres copias de los `.tf`
- Un arreglo del security group hay que copiarlo a las tres

Una carpeta de modulos compartida

```
module "api" {
  source = "../../modulos/api"

  nombre_entorno = var.nombre_entorno
  numero_tareas  = var.numero_tareas
  bucket_datos   = var.bucket_datos
}
```

> Una unica carpeta de modulos, asi reducimos algo la duplicidad

Qué estructura usar

Carpetas por entorno

Cuando las necesidades del proyecto lo demanden

Uso de `.tfvars`

Es el estandar

> En el laboratorio, al tener un solo entorno usaremos un terraform.tfvars para practicas el uso de estos ficheros

AGENDA

3 · Lock del estado

1. Ficheros tfvars
2. Gestión de entornos
3. **Lock del estado**

Lock del estado

Dos `apply` sobre el mismo estado

Sin lock

- Los dos leen el mismo mapa y escriben el fichero
- Gana el que termina el último
- El otro deja objetos que el estado ya no recuerda

Con lock

- El primero lo crea antes de escribir
- El segundo se para
- Al salir, el primero lo borra

> El lock es por fichero de estado

Dónde está

El lock es por fichero de estado. La key de un alumno no bloquea la de otro, ni la del profesor.

Estado local

`.terraform.tfstate.lock.info`, junto al estado.

S3

S3, `use_lockfile = true`. Objeto `<key>.tflock`. Terraform ≥ 1.10.

Dynamo

Una tabla de DynamoDB.

Si ya está tomado

Por defecto

Terraform falla al momento, con el id del lock.

`-lock-timeout=5m`

Reintenta ese tiempo y sigue si el otro proceso suelta el lock.

Lock del estado

Cuándo se queda pillado

Se corta

El portátil se cierra o cae la red a mitad del `apply`.

El proceso muere

`SIGKILL` o se cierra la terminal. Un Ctrl+C normal sí intenta soltarlo.

CI

El trabajo se cancela por tiempo.

> Es un JSON corto. No suele corromperse: se queda tomado.

Lock del estado

El error trae un ID

```
Error: Error acquiring the state lock

Lock Info:
  ID:        7c8e1a2b-4f0d-4a1e-9c3b-6d5e4f3a2b1c
  Operation: OperationTypeApply
  Who:       ana@portatil
  Created:   2026-10-15 09:12:03 +0000 UTC
```

Lock del estado

Lock vivo o lock pillado

Está trabajando

- `Who` eres tú, `Created` es de hace unos segundos
- Hay un `apply` visible en otra terminal o en CI

Está pillado

- El proceso ya no existe
- El `Created` es de ayer, o de un job cancelado

Lock del estado

Soltar uno que ya no corre

1. **Leer**
   `Who` y `Created`
2. **Comprobar**
   No hay un Terraform con ese estado
3. **force-unlock**
   El ID del error
4. **plan**
   El diff real, no el error de lock

El comando

Desde el mismo directorio y con las mismas credenciales. Terraform pide confirmación.

```
terraform force-unlock 7c8e1a2b-4f0d-4a1e-9c3b-6d5e4f3a2b1c
```

> Soltar el lock de un `apply` que sigue en marcha deja a los dos escribir. El estado puede quedar por detrás de AWS.

Takeaways

1. Un tfvars cambia valores. Varios entornos implican varios estados: el mismo estado hace que el apply de pro destruya lo de dev.
2. Mismos recursos y distinta configuración: un `main.tf` y un tfvars por entorno. Es el caso habitual.
3. Recursos distintos: una carpeta por entorno. Un módulo compartido reduce las copias.
4. El laboratorio es un solo entorno y un `terraform.tfvars` .
5. El lock es por fichero de estado.

Siguiente paso: laboratorio final.